

Unit 04: Requirement Specification

CONTENTS

Objective

Introduction

4.1 Problem Analysis

4.2 Characteristics of a Good SRS

4.3 Components of SRS

4.4 Structure of Document

4.5 Properties of a good SRS document

4.6 Functional and non-functional requirement

4.7 Modeling Techniques

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objective

After studying this unit, you will be able to:

- Discuss problem analysis
- Explain software requirement specification
- Describe the characteristics and components of SRS document.
- Explain software requirement specification language
- Discuss the structure of document

Introduction

As a result of the analysis, the Software Requirements Specification is created. Requirements analysis is a software engineering task that connects the dots between system and software engineering. Software engineers can utilize requirements analysis to fine-tune software allocation and construct models of data, function, and behavioral domains that will be used by the software. It gives the designer the knowledge he or she needs to create data, architectural, interface, and component-level designs. Finally, it allows both the developer and the customer to evaluate the quality of the software once it has been produced. Software requirements analysis can be divided into five areas of effort: Problem recognition, Evaluation and synthesis, Modeling, Specification, and Review. Initially, the analyst studies the system specification and the software project plan to understand the system as a whole in relation to the software and to review the scope of the software as per the planning estimates. After this the communication for analysis is established to ensure problem recognition.

The SRS (Software Requirements Specification) is a document that specifies specifications for a specific software product, programme, or collection of programmes that execute a specified function in a specified environment. A Software Requirements Specification (SRS) is a detailed description of a software system to be developed, including both functional and non-functional requirements. The customer could write the SRS. It must cater to the users' requirements and expectations. SRS could be written by the system's creator. It has a separate goal and emphasises the difference between the developer and the customer.

The analyst begins the requirements gathering and analysis process by obtaining all relevant information from the customer that could be utilised to establish the system's requirements. He then examines the data gathered to gain a clear and full understanding of the product that will be built, in order to eliminate any ambiguities or inconsistencies in the customer's initial impression of the problem. In order to have a good understanding of the problem, the analyst should be able to answer the following basic questions about the project.

- What is the problem?
- Why is it important to solve the problem?
- What are the possible solutions to the problem?
- What exactly are the data input to the system and what exactly are the data output by the system?
- What are the likely complexities that might arise while solving the problem?
- If there are external software or hardware with which the developed software has to interface, then what exactly would the data interchange formats with the external system be?

After the analyst has a thorough understanding of the customer's actual requirements, he moves on to identifying and resolving the numerous requirements issues. Anomalies, inconsistencies, and incompleteness are the most crucial requirements problems for the analyst to uncover and eliminate. When the analyst discovers any contradictions, anomalies, or gaps in the requirements, he remedies them by having additional discussions with the end users and customers.

4.1 Problem Analysis

The basic aim of problem analysis is to obtain a clear understanding of the needs of the clients and the users, what exactly is desired from the software, and what the constraints on the solution are. Frequently the client and the users do not understand or know all their needs, because the potential of the new system is often not fully appreciated. The analysts have to ensure that the real needs of the clients and the users are uncovered, even if they don't know them clearly. That is, the analysts are not just collecting and organizing information about the client's organization and its processes, but they also act as consultants who play an active role of helping the clients and users identify their needs.

The basic principle used in analysis is the same as in any complex task: divide and conquer. That is, partition the problem into sub problems and then try to understand each sub problem and its relationship to other sub problems in an effort to understand the total problem. The concepts of state and projection can sometimes also be used effectively in the partitioning process. A state of a system represents some conditions about the system. Frequently, when using state, a system is first viewed as operating in one of the several possible states, and then a detailed analysis is performed for each state.

Problem Analysis consists of the following approaches:

1. **Informal Approach:** An informal approach is one where no defined methodology is used to analyse. The information about the system is attained by interaction with the questionnaires, client, end users, study of existing documents, brainstorming, etc. The informal approach is used extensively to analysis and can be relatively suitable as conceptual modeling-based approaches repeatedly do not model all facets of the problem and are not always suitable for all the problems. As the Software Requirements Specification (SRS) is to be authorised and the feedback from the validation activity may necessitate for further analysis or requirement. Selecting an informal approach to analysis is not very uncertain – the errors that could be presented are not essentially heading for slip by the prerequisites phase. Therefore such approaches might be the utmost useful approach to analysis in some conditions.
2. **Shadowing:** Shadowing provides an effective means to discover what is currently being done in a business. Shadowing is a good way to get an idea of what a person does on a day-to-day basis. However, you might not be able to observe all the tasks during shadowing because more than likely the person will not, during that session, perform all the tasks he or she is assigned. For example, accounting people might create reports at the end of the month, developers might create status reports on a weekly basis, and management might schedule status meetings only biweekly. In addition, you can also learn the purpose of performing a specific task. To gather as much information as possible, you need to encourage the user to explain the reasons for performing a task in as much

detail as possible. Shadowing can be both passive and active. When performing passive shadowing, you observe the user and listen to any explanations that the user might provide. When performing active shadowing, you ask questions as the user explains events and activities. You might also be given the opportunity to perform some of the tasks, at the user's discretion.

3. **Interviews:** In interview is a one-on-one meeting between a member of the project team and a user. The quality of the information a team gathers depends on the skills of both the interviewer and the interviewee. An interviewer can learn a great deal about the difficulties and limitations of the current solution. Interviews provide the opportunity to ask a wide range of questions about topics that you cannot observe by means of shadowing. Shadowing is not the best option for gathering information about tasks such as management-level activities; long-term activities those span weeks, months, or years; or processes that require little or no human intervention. An example of a process that requires no human intervention is the automatic bill paying service provided by financial institutions. For gathering information about such activities and processes, you need to conduct interviews.

4.2 Characteristics of a Good SRS

An SRS should have the following characteristics:

Correct: An SRS is said to be correct if it contains the requirements that the software should meet. The correctness cannot be measured through a tool or a procedure.

Unambiguous: An SRS is unambiguous only if the requirements contained in it have only one interpretation. It must be unambiguous to both who create and who use it. However, the two groups may use different languages for description. But the language should be a natural language that can be easily understood by both the groups.

Complete: An SRS is complete if it includes the following:

All significant requirements related to functionality, performance, design, attributes or external interfaces, Definition of the software responses to all possible input classes in all possible situations, Full references and labels to figures, tables, diagrams and the definition of all terms and units of measure.

Consistent: An SRS is consistent if no subset of a requirement defines it in entirety. The conflicts can be of three types:

- Characteristics of real-world objects.
- Logical or temporal conflicts.
- The same real world object defined in more than one places in different terminologies.

Ranked for Importance and Stability: An SRS is ranked for importance and stability if each requirement contained in it has an identifier indicating, its importance or stability of that requirement, another way to implement the importance of requirements is to classify them as essential conditional and optional.

Verifiable: An SRS is verifiable if every requirement contained in it is verifiable i.e. there exists some defined method to judge if a software meets all the requirements. Non verifiable requirements should be removed or revised.

- **Modifiable:** An SRS is modifiable if its styles and structures can be retained easily, completely and consistently while changing a requirement.
- **Traceable:** An SRS is traceable if each of its requirements is clear and can be referenced in future development or enhancement documentation. Two types of traceability are recommended:
 - **Backward Traceability:** This depends upon each requirement explicitly referencing its source in earlier documents.
 - **Forward Traceability:** This depends upon each requirement in SRS having a unique name or reference number.
- **Backward Traceability:** This depends upon each requirement explicitly referencing its source in earlier documents.
- **Forward Traceability:** This depends upon each requirement in SRS having a unique name or reference number.

Design Independence: There should be an option to choose from multiple design alternatives for the final system. More specifically, the SRS should not include any implementation details.

Testability: A SRS should be written in such a way that it is easy to generate test cases and test plans from the document.

Understandable by the customer: An end user maybe an expert in his/her specific domain but might not be an expert in computer science. Hence, the use of formal notations and symbols should be avoided to as much extent as possible. The language should be kept easy and clear.

Right level of abstraction: If the SRS is written for the requirements phase, the details should be explained explicitly. Whereas, for a feasibility study, fewer details can be used hence, the level of abstraction varies according to the purpose of the SRS.

4.3 Components of SRS

Completeness of specifications is difficult to achieve and even more difficult to verify. Having guidelines about what different things an SRS should specify will help in completely specifying the requirements. The basic issues an SRS must address are:

Functionality

Performance

Design constraints imposed on an implementation

External interfaces

Functionality

Here functional requirements are to be specified. It should specify which outputs should be produced from the given input. For each functional requirement, a detailed description of all the inputs, their sources, range of valid inputs, the units of measure are to be specified. All the operation to be performed on input should also be specified.

Performance requirements

In this component of SRS all the performance constraints on the system should be specified such as response time, throughput constraints, number of terminals to be supported, number of simultaneous users to be supported etc.

Design constraints

Here design constraints such as standard compliance, hardware limitations, Reliability, and security should be specified. There may be a requirement that system will have to use some existing hardware, limited primary and/or secondary memory. So it is a constraint on the designer. There may be some standards of the organization that should be obeyed such as the format of reports. Security requirements may be particularly significant in defense systems.

It imposes a restriction sometimes on the use of some commands, control access to data; require the use of passwords and cryptography techniques etc

External Interface requirements

Software has to interact with people, hardware, and other software. All these interfaces should be specified. User interface has become a very important issue now a day. So the characteristics of user interface should be precisely specified and should be verifiable.



If there is a formula for computing the output, it should be specified.

An important part of the specification is the system behavior in abnormal situations, like invalid input (which can occur in many ways) or error during computation. The functional requirement must clearly state what the system should do if such situations occur. Specifically, it should specify the behavior of the system for invalid inputs and invalid outputs. Furthermore, behavior for situations where the input is valid but the normal operation cannot be performed should also be specified.



An example of this situation is a reservation system, where a reservation cannot be made even for a valid request if there is no availability.

In short, the system behavior for all foreseen inputs and all foreseen system states should be specified.

The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution characteristics of the system. These include requirements like the number of terminals to be supported, the number of simultaneous users to be supported, and the number of files that the system has to process and their sizes. These are also called capacity requirements of the system.

Dynamic requirements specify constraints on the execution behavior of the system. These typically include response time and throughput constraints on the system. Response time is the expected time for the completion of an operation under specified circumstances. Throughput is the expected number of operations that can be performed in a unit time.



The SRS may specify the number of transactions that must be processed per unit time, or what the response time for a particular command should be. Acceptable ranges of the different performance parameters should be specified, as well as acceptable performance for both normal and peak workload conditions.

All of these requirements should be stated in measurable terms. Requirements such as “response time should be good” or the system must be able to “process all the transactions quickly” are not desirable because they are imprecise and not verifiable. Instead, statements like “the response time of command x should be less than one second 90% of the times” or “a transaction should be processed in less than one second 98% of the times” should be used to declare performance specifications.

There are a number of factors in the client’s environment that may restrict the choices of a designer leading to design constraints. Such factors include standards that must be followed, resource limits, operating environment, reliability and security requirements, and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.



Some examples of these are:

Standards Compliance: This specifies the requirements for the standards the system must follow. The standards may include the report format and accounting procedures. There may be audit requirements which may require logging of operations.

Hardware Limitations: The software may have to operate on some existing or Notes predetermined hardware, thus imposing restrictions on the design. Hardware limitations can include the type of machines to be used, operating system available on the system, languages supported, and limits on primary and secondary storage. Reliability and Fault

Tolerance: Fault tolerance requirements can place a major constraint on how the system is to be designed, as they make the system more complex and expensive. Recovery requirements are often an integral part here, detailing what the system should do if some failure occurs to ensure certain properties.

Security: Security requirements are becoming increasingly important. These requirements place restrictions on the use of certain commands, control access to data, provide different kinds of access requirements for different people, require the use of passwords and cryptography techniques, and maintain a log of activities in the system. They may also require proper assessment of security threats, proper programming techniques, and use of tools to detect flaws like buffer overflow.

In the external interface specification part, all the interactions of the software with people, hardware, and other software should be clearly specified. For the user interface, the characteristics of each user interface of the software product should be specified. User interface is becoming increasingly important and must be given proper attention. A preliminary user manual should be created with all user commands, screen formats, an explanation of how the system will appear to the user, and feedback and error messages. Like other specifications, these requirements should be precise and verifiable. So, a statement like “the system should be user friendly” should be avoided

and statements like “commands should be no longer than six characters” or “command names should reflect the function they perform” used.

For hardware interface requirements, the SRS should specify the logical characteristics of each interface between the software product and the hardware components. If the software is to execute on existing hardware or on predetermined hardware, all the characteristics of the hardware, including memory restrictions, should be specified. In addition, the current use and load characteristics of the hardware should be given.

The interface requirement should specify the interface with other software the system will use or that will use the system. This includes the interface with the operating system and other applications.

4.4 Structure of Document

The Institute of Electrical and Electronics Engineers (IEEE) has published guidelines and standards to organize an SRS document. There is no single method that is suitable for all projects.

The different ways are proposed to structure the SRS. Although the first two sections are the same, the third section containing the “specific requirements” can differ.

Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies

3. Specific Requirements

4.5 Properties of a good SRS document

The important properties of a good SRS document are the following:

Concise: The SRS document should be concise and at the same time unambiguous, consistent, and complete. Verbose and irrelevant descriptions reduce readability and also increase error possibilities.

Structured: It should be well-structured. A well-structured document is easy to understand and modify. In practice, the SRS document undergoes several revisions to cope up with the customer requirements. Often, the customer requirements evolve over a period of time. Therefore, in order to make the modifications to the SRS document easy, it is important to make the document well-structured.

Black-box view: It should only specify what the system should do and refrain from stating how to do these. This means that the SRS document should specify the external behavior of the system and not discuss the implementation issues. The SRS document should view the system to be developed as black box, and should specify the externally visible behavior of the system. For this reason, the SRS document is also called the black-box specification of a system.

Conceptual integrity: It should show conceptual integrity so that the reader can easily understand it.

Response to undesired events: It should characterize acceptable responses to undesired events. These are called system response to exceptional conditions.

Verifiable: All requirements of the system as documented in the SRS document should be verifiable. This means that it should be possible to determine whether or not requirements have been met in an implementation.

Problems without a SRS document

The important problems that an organization would face if it does not develop a SRS document are as follows:

Without developing the SRS document, the system would not be implemented according to customer needs.

Software developers would not know whether what they are developing is what exactly required by the customer.

Without SRS document, it will be very much difficult for the maintenance engineers to understand the functionality of the system.

It will be very much difficult for user document writers to write the users' manuals properly without understanding the SRS document.

Problems with an unstructured specification

- It would be very much difficult to understand that document.
- It would be very much difficult to modify that document.
- Conceptual integrity in that document would not be shown.
- The SRS document might be unambiguous and inconsistent.

4.6 Functional and non-functional requirement

Functional requirement

A functional requirement defines a system or its component. It describes the functions software must perform.

A function is nothing but inputs, its behavior, and outputs. It can be a calculation, data manipulation, business process, user interaction, or any other specific functionality which defines what function a system is likely to perform.

A functional requirement will describe a particular behaviour of function of the system when certain conditions are met, for example: Send email when input is not given on time, any Requirement Which Specifies "What the System Should Do".

Depend on the type of software, expected users and the type of system where the software is used. Functional system requirements should describe the system services in detail.

Functional requirements include:

- Business Rules
- Transaction corrections, adjustments and cancellations
- Administrative functions
- Authentication
- Authorization levels
- Audit Tracking
- External Interfaces
- Certification Requirements
- Reporting Requirements
- Historical Data
- Legal or Regulatory Requirements

Business requirements: They contain the ultimate goal, such as an order system, an online catalogue, or a physical product. It can also include things like approval workflows and authorization levels.

Administrative functions. They are the routine things the system will do, such as reporting.

User requirements: They are what the user of the system can do, such as place an order or browse the online catalogue.

System requirements: These are things like software and hardware specifications, system responses, or system actions.

Functional Requirements Examples:

Only Managerial level employees have the right to view revenue data.

The software system should be integrated with banking API

The Sales system should allow users to record customer's sales

Authentication of a user when he/she tries to log into the system

System shutdown in the case of a cyber-attack

Verification email is sent to user whenever he/she registers for the first time on some software system.

Advantages of Functional Requirement

- A functional requirement document helps you to define the functionality of a system or one of its subsystems.
- Functional requirements along with requirement analysis help identify missing requirements. They help clearly define the expected system service and behavior.
- Errors caught in the Functional requirement gathering stage are the cheapest to fix.
- Support user goals, tasks, or activities for easy project management.
- Functional requirement can be expressed in Use Case form or user story as they exhibit externally visible functional behavior.
- Helps you to check whether the application is providing all the functionalities that were mentioned in the functional requirement of that application.

Non Functional Requirement

A non-functional requirement defines the quality attribute of a software system. They represent a set of standards used to judge the specific operation of a system.

Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards. A non-functional requirement is essential to ensure the usability and effectiveness of the entire software system. A non-functional requirement often applies to the system as a whole rather than individual features or services.

Non-functional Requirements allows you to impose constraints or restrictions on the design of the system across the various agile backlogs. Nonfunctional requirements deal with the characteristics of the system which cannot be expressed as functions - such as the maintainability of the system, portability of the system, usability of the system, etc.

Non Functional Requirement includes:

Portability

Security

Maintainability

Reliability

Scalability

Performance

Reusability

Flexibility

Reliability/Availability: What are the uptime requirements? Does it need to function 24/7/365?

Performance: How fast does it need to operate?

Usability: This focuses on the appearance of the user interface and how people interact with it. What colour are the screens? How big are the buttons?

Scalability: As needs grow, can the system handle it? For physical installations, this includes spare hardware or space to install it in the future.

Supportability: Is support provided in-house or is remote accessibility for external resources required?

Security: What are the security requirements, both for the physical installation and from a cyber-perspective?



Non-functional requirements

- A website should be capable enough to handle 20 million users with affecting its performance.
- The software should be portable. So moving from one OS to other OS does not create any problem.
- Privacy of information, the export of restricted technologies, intellectual property rights, etc. should be audited.
- Emails should be sent with a latency of no greater than 10 hours.
- Each request should be processed within predefined time
- The site should load in 5 seconds when the number of simultaneous users are > 15000

Advantages of Non-functional requirements

They ensure the reliability, availability, and performance of the software system

They ensure good user experience and ease of operating the software.

The nonfunctional requirements ensure the software system follow legal and compliance rules.

They help in formulating security policy of the software system.

Identifying functional requirements from a problem description

The high-level functional requirements often need to be identified either from an informal problem description document or from a conceptual understanding of the problem. Each high level requirement characterizes a way of system usage by some user to perform some meaningful piece of work. There can be many types of users of a system and their requirements from the system may be very different. So, it is often useful to identify the different types of users who might use the system and then try to identify the requirements from each user's perspective.



Consider the case of the library system, where –

F1: Search Book function

Input: an author's name

Output: details of the author's books and the location of these books in the library

So the function Search Book (F1) takes the author's name and transforms it into book details.

Functional requirements actually describe a set of high-level requirements, where each high-level requirement takes some data from the user and provides some data to the user as an output. Also each high-level requirement might consist of several other functions.

Documenting functional requirements

For documenting the functional requirements, we need to specify the set of functionalities supported by the system. A function can be specified by identifying the state at which the data is to be input to the system, its input data domain, the output data domain, and the type of processing to be carried on the input data to obtain the output data. Let us first try to document the withdraw-cash function of an ATM (Automated Teller Machine) system. The withdraw-cash is a high-level requirement. It has several sub-requirements corresponding to the different user interactions. These different interaction sequences capture the different scenarios.



Withdraw Cash from ATM

R1: withdraw cash

Description: The withdraw cash function first determines the type of account that the user has and the account number from which the user wishes to withdraw cash. It checks the balance to determine whether the requested amount is available in the account. If enough balance is available, it outputs the required cash; otherwise it generates an error message.

R1.1 select withdraw amount option

Input: "withdraw amount" option

Output: user prompted to enter the account type

R1.2: select account type

Input: user option

Output: prompt to enter amount

R1.3: get required amount

Input: amount to be withdrawn in integer values greater than 100 and less than 10,000 in multiples of 100.

Output: The requested cash and printed transaction statement.

Processing: the amount is debited from the user's account if sufficient balance is available, otherwise an error message displayed

4.7 Modeling Techniques

The three most important modeling techniques used in analyzing and building information systems are:

Data Flow Diagramming (DFDs): Data Flow Diagrams (DFDs) model events and processes (i.e. activities which transform data) within a system. DFDs examine how data flows into, out of, and within the system. (Note: 'data' can be understood as any 'thing' (e.g.: raw materials, filed information, ideas, etc.) which is processed within the system.

Logical Data Structure modeling (LDSs): Logical Data Structures (LDSs) represent a system's information and data in another way. LDSs map the underlying data structures as entity types, entity attributes, and the relationships between the entities.

Entity Life Histories (ELHs): Entity Life Histories (ELHs) describe the changes which happen to 'things' (entities) within the system

These three techniques are common to many methodologies and are widely used in system analysis. Notation and graphics style may vary across methodologies, but the underlying principles are generally the same. In SSADM (Structured Systems Analysis and Design Methodology - which has for a number of years been widely used in the UK) systems analysts and modelers use the above techniques to build up three, inter-related, views of the target system, which are cross-checked for consistency.

Summary

- The basic aim of problem analysis is to obtain a clear understanding of the needs of the clients and the users, what exactly is desired from the software, and what the constraints on the solution are.
- Data flow diagrams (also called data flow graphs) are commonly used during problem analysis.
- The SRS (Software Requirements Specification) contains specifications for a particular software product, program or a set of programs that perform a particular function in a specific environment.
- The performance requirements part of an SRS specifies the performance constraints on the software system.

- Unlike formal language that allows developers and designers some latitude, the natural language of software requirements specifications must be exact, without ambiguity, and precise.

Keywords

DFD: A DFD shows the flow of data through a system.

Evolutionary Prototyping: It uses the prototype as the first part of an analysis activity that will be continued into design and construction.

Object-Oriented Software Engineering (OOSE): It is a software design technique that is used in software design in object-oriented programming.

Problem Analysis: The basic aim of problem analysis is to obtain a clear understanding of the needs of the clients and the users, what exactly is desired from the software, and what the constraints on the solution are.

Requirements Analysis: It is a software engineering task that bridges the gap between system level engineering and software design.

SRS: Software Requirements Specification contains specifications for a particular software product, program or a set of programs that perform a particular function in a specific environment.

Self Assessment

1. Functional and non-functional requirement are the part of ____
 - A. Design phase
 - B. SRS
 - C. Testing
 - D. Maintenance
2. Functional requirements are____
 - A. Defines a system or its component
 - B. Describe a particular behavior or function of the system when certain conditions are met
 - C. "What the System Should Do"
 - D. All of above
3. Functional requirements not include ____
 - A. Business Rules
 - B. Administrative functions
 - C. Reusability
 - D. Authentication
4. Audit Tracking, External Interfaces and historical data is part of ____
 - A. Nonfunctional requirement
 - B. Functional requirement
 - C. All of the above
5. Nonfunctional requirements are ____
 - A. Defines the quality attribute of a software system

- B. Represent a set of standards used to judge the specific operation of a system
- C. Essential to ensure the usability and effectiveness of the entire software system
- D. All of above

6. Non Functional Requirement include

- A. Scalability
- B. Performance
- C. Reusability
- D. All of above

7. Each request should be processed within predefined time is _____

- A. Nonfunctional requirement
- B. Functional requirement
- C. All of the above

8. SRS stands for _____

- A. System Requirements Specification
- B. Software Requirements Specification
- C. Software Requirements support
- D. None of above

9. Software Requirements Specification is _____

- A. Developed with its functional and non-functional requirements
- B. Must address the needs and expectations of the users
- C. It serves a different purpose and mentions the contrast between the customer and the developer.
- D. All of above

10. Characteristics of a SRS _____

- A. Correct
- B. Unambiguous
- C. Complete
- D. All of above

11. As per IEEE which is not a part of Structure of Document

- A. Purpose
- B. Scope
- C. Testing
- D. Product Functions

12. The SRS document is also known as _____ specification

- A. Black-box

- B. White-box
- C. Grey-box
- D. None of above

13. Which of the following is not included in SRS?

- A. Functionality
- B. Performance
- C. Design solutions
- D. External Interfaces

14. What is the first step of requirement elicitation?

- A. Identifying Stakeholder
- B. Listing out Requirements
- C. Requirements Gathering
- D. All of above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. D | 3. C | 4. B | 5. D |
| 6. D | 7. A | 8. B | 9. D | 10. D |
| 11. C | 12. A | 13. C | 14. D | |

Review Questions

1. What is the aim of problem analysis? Discuss.
2. Describe the different approaches used in problem analysis.
3. Explain the concept of data flow in software requirements.
4. Discuss the concept of object oriented modeling.
5. Describe the techniques used for conducting rapid prototyping.
6. Explain Software Requirements Specification (SRS). Also discuss the characteristics of SRS.
7. Discuss the different components of SRS.
8. Discuss the language quality characteristics of an SRS.
9. Discuss how to represent the Structure of SRS document?



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner's Approach, 5th Edition, Tata McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education



- <http://softwareengineeringhub.blogspot.in/2010/03/system-requirementsspecification.html>
- <http://computersciencesource.wordpress.com/2010/03/14/softwareengineering-object-oriented-modelling/>
- <http://www.software-requirements.tuffley.com/sample.pdf>
- <https://www.geeksforgeeks.org/>
- <https://educatech.in/>